Laporan Tugas Kecil 2 IF2211 Strategi Algoritma

Voxelization Objek 3D menggunakan Octree

Semester II Tahun Ajaran 2025/2026

Disusun oleh:

Fazri Arrashyi Putra (13524127)
Dika Pramudya Nugraha (13524132)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

Kata Pengantar

Dokumen Laporan Tugas Besar 1 ini disusun mahasiswa Teknik Informatika angkatan 2024. Adapun susunan anggota kelompok kami secara detail sebagai berikut:

1. **Fazri** Arrashyi Putra (NIM: 13524**127**)
2. **Dika** Pramudya Nugraha (NIM: 13524**132**)

Daftar Isi

Kata Pengantar.....1
Daftar Isi.....1
BAB 1
DASAR TEORI.....3
1.1 Dasar Teori Octree.....3
1.2 Algoritma Divide and Conquer.....4
1.3 Voxelization.....5
BAB 2
IMPLEMENTASI ALGORITMA DALAM BAHASA C++.....7
2.1 File types.h.....7
2.2 File octree.h.....8
2.3 File octree.cpp.....9
2.4 File obj_io.h.....10
2.5 File obj_io.cpp.....11
2.6 File voxelizer.h.....12
2.7 File voxelizer.cpp.....13
2.8 File viewer.h.....13
2.9 File viewer.cpp.....14
2.10 File main.cpp.....16
BAB 3
SOURCE CODE PROGRAM.....17
3.1 Repository Program.....17
3.2 Struktur Source Code.....17
3.2.1 types.h.....17
3.2.2 octree.h.....18
3.2.3 octree.cpp (Implementasi Pembagian Ruang Menjadi 8 Child).....19
3.2.4 octree.cpp (Implementasi Uji Irisan Triangle vs Box).....19
3.2.5 octree.cpp (Implementasi Rekursi Utama Octree dan Pruning).....20
3.2.6 octree.cpp (Implementasi Inisialisasi Data Kandidat Sebelum Rekursi).....22
3.2.7 Octree.cpp (Implementasi Ekstraksi Voxel dari Leaf).....23
3.2.8 Voxelizer.h.....23
3.2.9 voxelizer.cpp.....24
3.2.10 main.cpp.....25
BAB 4
ANALISIS ALGORITMA.....27
4.1 Analisis Kompleksitas Waktu.....27

4.2 Analisis Kompleksitas Ruang....27
4.3 Pengaruh Tingkat Kedalaman (Depth) terhadap Kinerja dan Resolusi....27
4.4 Analisis Kinerja Konkurensi (Concurrency)....28
BAB 5
MASUKAN DAN LUARAN PROGRAM....29
5.1 Test Case 1 (cow.obj)....29
5.2 Test Case 2 (line.obj)....30
5.3 Test Case 3 (pumpkin.obj)....31
5.4 Test Case 4 (teapot.obj)....32
5.5 Test Case 5 (empty-test.obj)....33
LAMPIRAN....34
PERNYATAAN TIDAK MELAKUKAN KECURANGAN....35
DAFTAR PUSTAKA....36

BAB 1

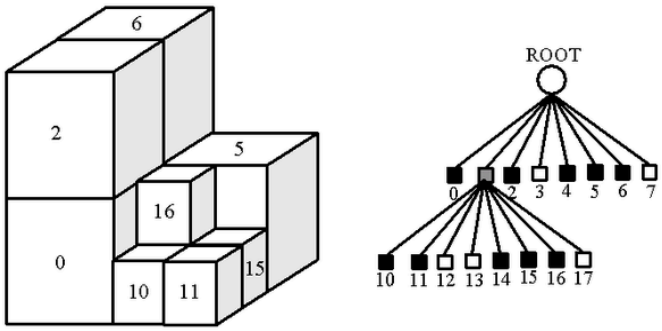
DASAR TEORI

1.1 Dasar Teori Octree

Octree merupakan salah satu bentuk struktur data berbasis pohon (*tree*) hierarkis yang dirancang secara khusus untuk mempartisi ruang tiga dimensi (3D). Konsep ini pada dasarnya adalah bentuk ekspansi spasial dari struktur data *quadtree* yang beroperasi pada bidang dua dimensi. Dalam arsitektur ruang *octree*, setiap simpul (*node*) internal secara konsisten memiliki tepat delapan simpul anak. Pendekatan pembagian ruang ini sangat esensial dalam pemrosesan grafika komputer karena kemampuannya untuk mengelola volume data spasial yang masif. Dibandingkan dengan merepresentasikan seluruh ruang tiga dimensi sebagai kumpulan matriks padat yang boros memori, struktur ini bekerja secara efisien dengan cara membagi sebuah lingkup besar menjadi subsistem-subsistem berukuran lebih kecil secara terstruktur berdasarkan tingkat keberisian ruangnya.

Proses partisi spasial ini mendefinisikan sebuah ruang kubus utama, yang sering disebut sebagai *bounding box*, yang melingkupi keseluruhan area geometri objek secara utuh. Kotak pembatas utama bertindak sebagai *root* dari struktur pohon.Melalui mekanisme pembagian yang merata, ruang kubus utama tersebut dipotong tepat pada titik tengah dari masing-masing sumbu koordinat kartesian (*x, y, z*) secara simultan. Pemotongan tiga sumbu ini akan menghasilkan delapan ruang kubus baru yang volumenya identik secara presisi, yang dikenal dengan istilah *octant*. Setiap *octant* tersebut mewakili satu dari delapan area kuadran spasial dalam sistem koordinat tiga dimensi lokal milik simpul induknya.

Pada tahap selanjutnya, jika *octant* terdeteksi memiliki irisan dengan geometri objek atau mengandung poligon dari model aslinya, maka statusnya dianggap sebagai ruang spasial, dan *octant* tersebut akan dipotong kembali menjadi delapan kubus turunan yang ukurannya lebih kecil. Pemotongan ruang ini berjalan secara rekursi ke tingkat kedalaman (*depth*) yang terus meningkat. Sebaliknya, jika suatu area *octant* divalidasi sama sekali kosong atau sepenuhnya padat berada di dalam objek tanpa perpotongan dengan batasan permukaan, penelusuran rekursif pada cabang tersebut akan langsung dihentikan dan simpulnya dideklarasikan sebagai simpul daun. Metode ini berfungsi sebagai metode pemangkasan cabang untuk menekan kompleksitas waktu komputasi, karena area ruang yang tidak relevan dengan permukaan objek tidak perlu diproses lebih lanjut. Korelasi antara pembagian ruang tiga dimensi secara fisik dengan pemetaan graf pohon hierarkinya dapat diuraikan secara visual melalui gambar ilustrasi berikut.



Gambar 1. Visualisasi *Octree* terhadap 3D Model
Sumber:
https://www.researchgate.net/figure/a-Octants-b-Octree-Structure-c-Linear-Octree-Each-of-the-cubes-has-a-locational-code_fig1_221115507

Pada gambar 1 tersebut mengilustrasikan relasi langsung antara dekomposisi kubus secara fisik di sisi kiri dengan pembentukan simpul-simpul dalam graf pohon di sisi kanan. Pada struktur pohon tersebut, simpul akar terbagi menjadi delapan simpul anak tingkat pertama yang merepresentasikan selapan *octant* awal. Dapat diamati secara spesifik bahwa salah satu simpul pada percabangan pertama dipecah lebih lanjut menjadi simpul-simpul anak baru (indeks 10 hingga 17). Pemecahan ini merepresentasikan penelusuran rekursif pada area kubus yang tervalidasi memotong objek. Perbedaan ukuran pada balok-balok susunan di gambar kiri merepresentasikan variasi tingkat kedalaman simpul daun di dalam memori pohon *octree*, di mana representasi blok terkecil menunjukkan simpul yang berada pada kedalaman pemrosesan paling jauh.

Pada hasil akhirnya, landasan arsitektur hierarkis ini dimanfaatkan secara maksimal untuk merealisasikan proses *voxelization*. Melalui algoritma penelusuran pohon ini, bentuk dasar tiga dimensi yang umumnya direpresentasikan oleh sekumpulan titik koordinat dan jaring poligon dapat ditransformasikan ke dalam bentuk diskrit.

1.2 Algoritma Divide and Conquer

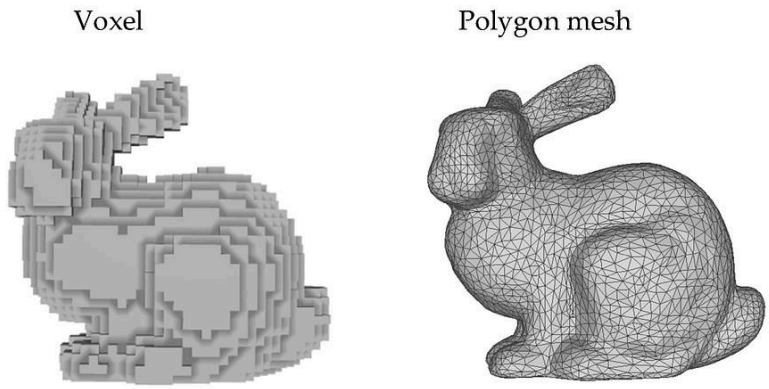
Divide and Conquer adalah strategi pemecahan masalah yang melibatkan pembagian masalah kompleks menjadi bagian-bagian yang lebih kecil dan mudah dikelola, menyelesaikan setiap bagian secara terpisah, dan kemudian menggabungkan solusi-solusi tersebut untuk memecahkan masalah aslinya. Metode ini melibatkan tiga langkah utama. Pertama, persoalan utama dibagi menjadi beberapa upa-persoalan yang memiliki kemiripan dengan persoalan aslinya, namun ukurannya lebih kecil. Idealnya, setiap upa-persoalan memiliki ukuran yang hampir sama. Kedua, setiap upa-persoalan diselesaikan secara langsung jika sudah berukuran kecil, atau secara rekursif jika masih berukuran besar. Terakhir, solusi dari masing-masing upa-persoalan digabungkan kembali sehingga

membentuk solusi dari persoalan utama. Metode ini berguna dalam menangani masalah kompleks dengan memecahkannya menjadi bagian-bagian yang lebih mudah diatasi, menyelesaikan masing-masing bagian, dan kemudian menggabungkan solusi-solusi tersebut untuk memperoleh solusi akhir.

Keunggulan dari algoritma *Divide and Conquer* terletak pada efisiensi penyelesaian masalah yang kompleks. Sebagai contoh, kompleksitas untuk perkalian dua matriks menggunakan metode biasa adalah $O(n^3)$, sedangkan menggunakan pendekatan divide and conquer seperti perkalian matriks Strassen memiliki kompleksitas $O(n^{2.8074})$. Keunggulan lainnya adalah kecocokan algoritma ini untuk sistem multiproses yang memungkinkan penerapan paralelisme untuk meningkatkan kinerja. Selain itu, algoritma ini menggunakan memori cache secara efisien, sehingga mengoptimalkan sumber daya memori yang tersedia.

1.3 Voxelization

Voxelization adalah proses transformasi representasi geometris dari sebuah objek tiga dimensi (3D), yang umumnya direpresentasikan oleh jaring-jaring poligon (mesh), menjadi representasi diskret berbasis voxel (elemen volume piksel). Voxel adalah satuan kubus terkecil dan seragam dalam ruang 3D, serupa dengan piksel dalam gambar 2D, yang menyimpan informasi keberadaan objek atau properti tertentu dalam volume spasial tersebut. Tujuan utama dari voxelization adalah untuk memfasilitasi pemrosesan spasial yang lebih efisien, analisis fisika, serta render grafis, terutama ketika berhadapan dengan data objek yang kompleks atau volumetrik.



Gambar 2 Visualisasi *voxelization*
Sumber: https://www.mdpi.com/1424-8220/22/18/7040

Dalam konteks penggunaan Octree (seperti yang dijelaskan pada Sub-bab 1.1), proses voxelization memanfaatkan struktur pohon hierarkis tersebut untuk secara efisien mengelola dan menentukan lokasi voxel yang terisi. Alih-alih memindai seluruh ruang 3D secara brute-force, Octree membagi ruang secara rekursif menjadi delapan sub-ruang (octant). Pembagian ini dihentikan pada tingkat kedalaman (depth) tertentu, di mana simpul daun pada Octree yang tervalidasi beririsan dengan permukaan objek (occupied) akan merepresentasikan satu atau lebih voxel yang mengisi volume objek tersebut. Pendekatan berbasis Octree ini secara signifikan mengurangi kebutuhan memori dan waktu komputasi, karena hanya ruang-ruang yang relevan dengan permukaan objek yang perlu dipartisi dan divisualisasikan sebagai voxel. Proses ini menghasilkan representasi objek yang terdiri dari kumpulan kubus diskret, di mana ukuran resolusi voxel ditentukan oleh kedalaman maksimum dari pohon Octree yang digunakan.

BAB 2

IMPLEMENTASI ALGORITMA DALAM BAHASA C++

2.1 File types.h

File ini berisi definisi berbagai struktur data utama yang digunakan untuk merepresentasikan titik, poligon, serta elemen-elemen penyusun pohon *octree* dalam keseluruhan proses *voxelization*.

Nama Struktur	Deskripsi
Vertex	Struktur data untuk merepresentasikan titik koordinat tiga dimensi (x, y, z) dari sebuah model dalam ruang spasial.
Face	Struktur data untuk menyimpan indeks tiga titik sudut (v1, v2, v3) yang saling terhubung untuk membentuk sebuah poligon segitiga pada permukaan objek 3D.
MyBoundingBox	Struktur data yang mendefinisikan batas ruang kubus (*bounding box*) dengan menyimpan informasi titik ekstrem minimum dan maksimumnya.
VoxelizationResult	Struktur data komprehensif untuk menampung seluruh hasil akhir dari proses *voxelization*. Struktur ini menyimpan sekumpulan data *vertex* dan *face* (baik dari model asli maupun hasil konversi), kotak pembatas utama, lokasi penyimpanan *file* keluaran, jumlah total *voxel* yang terbentuk, hingga perhitungan lama waktu eksekusi program.
OctreeStats	Struktur data untuk mencatat dan mengelola statistik pembentukan pohon *octree*, yang mencakup pencatatan jumlah simpul yang dibentuk dan dipangkas (*pruned*) pada setiap tingkat kedalaman. Struktur ini dilengkapi dengan mekanisme *mutex* untuk memastikan keamanan data (*thread-safe*) saat menerapkan konkurensi.
OctreeNode	Struktur data yang merepresentasikan satu simpul (*node*) individual di dalam pohon *octree*. Simpul ini menyimpan informasi mengenai ruang batasannya, tingkat kedalaman (*depth*), status apakah simpul tersebut adalah daun (*leaf*), status interseksi dengan objek (*occupied*), serta *pointer* yang mengarah ke delapan simpul anaknya.

2.2 File octree.h

File ini berisi deklarasi fungsi-fungsi yang bertanggung jawab atas perhitungan geometris spasial dan logika utama pembentukan pohon *octree* secara rekursif dalam proses *voxelization*.

Nama Fungsi	Deskripsi
computeBoundingBox	Fungsi ini berfungsi untuk menghitung dan mengembalikan batas ruang kubus (*bounding box*) utama yang secara presisi melingkupi keseluruhan titik koordinat dari model 3D masukan.
triangleBoxIntersect	Fungsi ini berfungsi untuk mengimplementasikan algoritma pengujian interseksi matematis untuk menentukan apakah sebuah poligon segitiga saling berpotongan dengan suatu kotak pembatas ruang.
meshIntersectsBox	Fungsi ini berfungsi untuk memeriksa secara komprehensif apakah terdapat bagian mana pun dari permukaan model objek 3D (berdasarkan kumpulan *vertices* dan *faces*) yang bersinggungan dengan ruang kotak pembatas yang sedang dievaluasi.
buildOctree	Fungsi ini merupakan fungsi yang mengeksekusi algoritma *Divide and Conquer* secara rekursif untuk membangun hierarki pohon *octree*. Fungsi ini memecah simpul menjadi delapan anak cabang baru apabila terdeteksi interseksi geometri, dan terus berlanjut hingga mencapai batas kedalaman maksimum (*max depth*).
extractVoxels	Fungsi yang menelusuri struktur pohon *octree* yang telah terbentuk untuk mengekstrak data *vertex* serta menyusun *face* dari simpul-simpul daun pada kedalaman maksimum, guna membentuk representasi *voxel* sebagai hasil akhir konversi.

2.3 File octree.cpp

File ini memuat implementasi dari seluruh fungsi yang dideklarasikan pada octree.h. File ini menangani perhitungan matematika spasial secara detail dan mengeksekusi logika *Divide and Conquer* untuk membangun pohon *octree*.

Nama Fungsi	Deskripsi
boxesOverlap	Fungsi *helper* yang mengecek apakah *bounding box* saling tumpang tindih.
childBox	Fungsi yang membagi sebuah kotak pembatas induk menjadi salah satu dari delapan *octant* berdasarkan indeks (0-7).
appendCube	Fungsi yang menyusun dan menambahkan delapan titik sudut serta 12 bidang segitiga dari sebuah kubus (*voxel*) ke dalam senarai titik dan wajah akhir.
triangleBounds	Fungsi untuk menghitung dan mengembalikan batas kotak pembatas yang paling pas untuk melingkupi sebuah poligon segitiga tunggal.
sub, cross, dot, absf	Kumpulan fungsi matematika dasar untuk melakukan operasi vektor tiga dimensi, meliputi pengurangan, perkalian silang (*cross product*), perkalian titik (*dot product*), dan nilai absolut.
satAxisOverlap	Fungsi bagian dari *Separating Axis Theorem* (SAT) untuk menguji apakah proyeksi sebuah segitiga dan sebuah kubus pada suatu sumbu tertentu saling tumpang tindih.
incrementCreated, incrementPruned	Fungsi utilitas untuk memperbarui data statistik *octree* secara aman (menggunakan *mutex*) dengan mencatat jumlah simpul yang berhasil dibuat atau dipangkas pada tingkat kedalaman tertentu.
buildOctreeFiltered	Fungsi rekursif inti yang melakukan proses *Divide and Conquer*. Fungsi ini menyaring poligon-poligon yang memotong simpul saat ini, membagi ruang menjadi delapan *octant*, dan memanggil dirinya sendiri untuk setiap anak cabang jika ditemukan interseksi, hingga mencapai batas kedalaman.
computeBoundingBox	Fungsi yang mengimplementasikan pencarian titik

	minimum dan maksimum dari seluruh koordinat model untuk membentuk kotak pembatas utama (*root box*).
triangleBoxIntersect	Fungsi yang mengimplementasikan algoritma interseksi Triangle-AABB (Axis-Aligned Bounding Box) menggunakan metode *Separating Axis Theorem* (SAT) secara lengkap.
meshIntersectsBox	Fungsi yang melakukan iterasi ke seluruh poligon wajah pada model dan memanggil *triangleBoxIntersect* untuk mengecek apakah ada bagian model yang memotong sebuah *bounding box*.
buildOctree	Fungsi yang menginisialisasi proses pembangunan *octree*. Fungsi ini menyiapkan data segitiga beserta batasnya, mencatat statistik simpul akar, dan memulai pemanggilan pertama ke fungsi rekursif *buildOctreeFiltered*.
extractVoxels	Fungsi rekursif yang menelusuri struktur pohon dari akar hingga simpul daun. Jika sebuah simpul diidentifikasi sebagai daun yang berinterseksi dengan objek (atau telah mencapai kedalaman maksimum), fungsi ini akan memanggil *appendCube* untuk mengekstrak simpul tersebut menjadi *voxel*.

2.4 File obj_io.h

File ini berisi deklarasi antarmuka untuk fungsi-fungsi yang menangani operasi masukan dan keluaran (I/O) dari sistem berkas, khususnya yang berkaitan dengan validasi, pembacaan, dan penulisan berkas dengan format .obj.

Nama Fungsi	Deskripsi
parseObj	Fungsi yang bertugas membaca berkas masukan dan memastikan formatnya valid dengan mengekstrak baris-baris yang berisikan nilai *v x y z* untuk titik koordinat dan *f i j k* untuk sisi poligon.
exportToObj	Fungsi yang mengembalikan hasil transformasi dengan menyimpan dan menyusun ulang sekumpulan titik dan bidang poligon *voxel* ke dalam sebuah berkas .obj keluaran yang baru.
getOutputFilename	Fungsi utilitas untuk merumuskan nama dan *path* di mana

	berkas .obj hasil konversi akan disimpan secara otomatis.
hasObjExtension	Fungsi validasi untuk memverifikasi bahwa *path* masukan yang diterima oleh program benar-benar mengarah pada sebuah berkas dengan ekstensi .obj yang tepat.
isVoxelizedObjName	Fungsi pemeriksa untuk mengidentifikasi apakah sebuah berkas sudah memiliki nama penanda keluaran konversi, biasanya digunakan untuk mencegah penimpaan atau pemrosesan ganda.
collectObjCandidates	Fungsi yang menelusuri suatu *path* atau direktori awal untuk mengumpulkan sekumpulan daftar berkas yang memenuhi syarat sebagai kandidat objek 3D untuk dikonversi.

2.5 File obj_io.cpp

File ini berisi implementasi teknis dari seluruh deklarasi antarmuka I/O yang terdapat pada obj_io.h. File ini memuat instruksi spesifik untuk memanipulasi *string*, berinteraksi dengan *file system*, serta melakukan pembacaan dan penulisan data geometris ke dalam memori sekunder.

Nama Fungsi	Deskripsi
parseObj	Fungsi yang mengimplementasikan logika pembacaan berkas baris demi baris menggunakan aliran data. Fungsi ini melakukan *parsing* secara spesifik pada baris yang memiliki awalan *v* untuk mengekstrak nilai *vertex* (*x, y, z*) dan awalan *f* untuk nilai *face* poligon. Fungsi ini juga memiliki penanganan (*error handling*) jika berkas gagal dibuka.
exportToObj	Fungsi yang mengimplementasikan proses penulisan struktur data objek 3D ke dalam ruang penyimpanan. Fungsi ini merangkai kembali senarai titik dan poligon hasil konversi dengan format baku .obj dan memberikan notifikasi mengenai lokasi akhir penyimpanan berkas.
getOutputFilename	Fungsi yang memanipulasi *string* dari *path* masukan untuk menghasilkan nama berkas keluaran secara otomatis. Fungsi ini mencari posisi titik ekstensi pada teks dan menyisipkan sufiks "-voxelized" sehingga menghasilkan nama akhir seperti *pumpkin-voxelized.obj*.

hasObjExtension	Fungsi yang melakukan operasi perbandingan *string* untuk memvalidasi apakah berkas yang diberikan diakhiri dengan ekstensi .obj. Proses ini mengonversi ekstensi ke huruf kecil terlebih dahulu untuk memastikan akurasi validasi.
isVoxelizedObjName	Fungsi yang melakukan pengecekan pada bagian akhir *string* nama berkas untuk memastikan keberadaan penanda "-voxelized.obj". Fungsi ini penting untuk mencegah iterasi pemrosesan ulang pada berkas yang sudah berhasil dikonversi sebelumnya.
collectObjCandidates	Fungsi yang memanfaatkan pustaka std::filesystem untuk menelusuri isi direktori secara otomatis. Fungsi ini memindai direktori kerja saat ini, subdirektori test, dan direktori induk masukan untuk mengumpulkan semua berkas .obj yang valid, membuang duplikat, dan mengurutkannya sebagai daftar kandidat konversi.

2.6 File voxelizer.h

File ini berisi deklarasi struktur konfigurasi dan fungsi utama yang bertindak sebagai pengatur jalannya keseluruhan proses *voxelization*. Berkas ini menjembatani data mentah masukan dengan algoritma inti pemrosesan *octree*.

Nama Struktur / Fungsi	Deskripsi
VoxelizerConfig	Struktur data yang berfungsi untuk menyimpan kumpulan parameter konfigurasi pengeksekusian *voxelization*. Struktur ini mencakup pengaturan batas kedalaman maksimum pohon *octree* (*maxDepth*), tanda untuk mengaktifkan pemrosesan secara konkuren atau asinkron (*useAsync*) untuk mengoptimalkan kinerja kinerja komputasi, serta variabel token pembatalan (*cancelToken*) untuk menghentikan proses algoritma secara aman di tengah eksekusi.
voxelizeMesh	Fungsi publik utama yang menjadi titik masuk untuk mengeksekusi transformasi model. Fungsi ini menerima sekumpulan titik koordinat (*vertices*) dan bidang poligon (*faces*) dari model 3D beserta konfigurasi pemrosesannya, lalu mengembalikan data terpadu berupa *VoxelizationResult* yang memuat seluruh *voxel* hasil konversi, statistik pembentukan simpul, dan perhitungan waktu eksekusi.

2.7 File voxelizer.cpp

File ini berisi implementasi dari fungsi pengatur yang dideklarasikan pada *voxelizer.h*. Berkas ini bertugas mengelola keseluruhan siklus hidup proses *voxelization*, mulai dari pencatatan metrik waktu, evaluasi struktur data masukan, hingga pengimplementasian eksekusi program secara paralel.

Nama Fungsi	Deskripsi
voxelizeMesh	Fungsi utama yang mengintegrasikan dan mengeksekusi seluruh alur transformasi model 3D secara komprehensif. Fungsi ini menginisialisasi pencatat waktu komputasi beresolusi tinggi, menghitung kotak pembatas utama, dan memulai pembangunan struktur hierarkis *octree*. Secara khusus, fungsi ini menerapkan mekanisme konkurensi menggunakan utilitas komputasi asinkron (*std::async*) untuk menjalankan pembangunan pohon secara paralel, implementasi ini ditujukan untuk memenuhi spesifikasi bonus optimasi kinerja pada tugas. Setelah struktur hierarkis terbentuk, fungsi ini akan memanggil prosedur ekstraksi *voxel*, menghitung estimasi total *voxel* padat (berdasarkan jumlah seluruh wajah poligon dibagi dua belas), dan mengembalikan rangkuman hasil eksekusi. Fungsi ini juga memiliki token pembatalan (*cancel token*) di setiap tahapannya untuk memastikan proses komputasi dapat diinterupsi dan dihentikan dengan aman tanpa kebocoran memori.

2.8 File viewer.h

Fiel viewer.h berfungsi sebagai antarmuka (interface) modul visualisasi dan GUI. Berkas ini mendefinisikan konfigurasi awal tampilan serta deklarasi fungsi utama untuk menjalankan viewer hasil voxelization dan antarmuka voxelizer.

Nama Struktur/Fungsi	Deskripsi
ViewerConfig	Struktur konfigurasi viewer yang menyimpan parameter awal jendela dan mode render. Struktur ini memudahkan pengaturan viewer tanpa mengubah implementasi internal.
ViewerConfig::width	Lebar jendela viewer (default 1280 piksel).
ViewerConfig::height	Tinggi jendela viewer (default 720 piksel).

ViewerConfig::startSmooth	Penanda apakah mode smooth shading aktif saat aplikasi dimulai (default false).
runViewer(const VoxelizationResult& result, const ViewerConfig& config)	Fungsi untuk menjalankan visualisasi hasil voxelization berdasarkan data VoxelizationResult dan konfigurasi awal ViewerConfig.
runVoxelizerGui(const vector<string>& objCandidates, int initialDepth, const string& initialPath, bool startSmooth)	Fungsi untuk menjalankan GUI voxelizer: menampilkan kandidat file OBJ, mengatur kedalaman awal voxelization, menetapkan path awal input, serta status awal smooth shading.

2.9 File viewer.cpp

viewer.cpp merupakan implementasi modul antarmuka visualisasi 3D dan GUI voxelizer. Berkas ini menangani seluruh alur dari pemuatan model OBJ, proses voxelisasi secara asynchronous, pembentukan mesh untuk ditampilkan, kontrol kamera orbit, hingga rendering informasi statistik hasil voxelisasi pada panel UI.

Nama Struktur/Fungsi	Deskripsi
BuildOutput	Struktur keluaran proses voxelisasi yang menyimpan status keberhasilan, pesan status, dan objek VoxelizationResult.
OriginalPreviewOutput	Struktur keluaran pemuatan preview model original, berisi status, pesan, path input, data vertices, faces, serta bounding box model.
toVec3, subtract, cross, normalize	Kumpulan utilitas vektor 3D untuk konversi data vertex dan perhitungan geometri (normalisasi, selisih vektor, dan cross product).
buildMeshFromFaces(...)	Membangun objek Mesh Raylib dari data vertices dan faces, dengan dukungan normal flat maupun smooth sesuai mode tampilan.
createLightingShader()	Membuat shader pencahayaan kustom (vertex + fragment shader) untuk menampilkan model 3D dengan efek ambient, diffuse, dan specular.
maxComponent(...)	Mengambil nilai terbesar dari tiga komponen dimensi,

	digunakan untuk menentukan skala scene dan batas zoom kamera.
baseName(...)	Mengekstrak nama file dari path untuk kebutuhan tampilan UI.
computeBoxFromVertices (...)	Menghitung bounding box minimum–maksimum dari sekumpulan vertex model.
loadOriginalPreview(...)	Memuat file OBJ original, melakukan validasi data mesh, dan menyiapkan data preview awal untuk ditampilkan.
processVoxelization(...)	Menjalankan proses voxelisasi mesh menggunakan konfigurasi kedalaman octree, mendukung pembatalan proses, serta mengekspor hasil ke file OBJ output.
measureTextWithCurrentFont, drawTextWithCurrentFont, drawWrappedTextWithCurrentFont	Utilitas rendering teks UI dengan dukungan font kustom, skala tampilan, dan pembungkusan teks otomatis.
drawButton, drawDropdown	Komponen UI interaktif untuk tombol dan dropdown pada panel kontrol voxelizer.
buildDepthStatsLines, compactStats, normalizePathForDisplay	Utilitas formatting statistik per kedalaman octree dan normalisasi path agar lebih terbaca di antarmuka.
runViewer(...)	Menjalankan viewer 3D hasil voxelisasi: inisialisasi window, kontrol kamera orbit, toggle mode smooth/flat, wireframe, grid, dan info overlay.
runVoxelizerGui(...)	Fungsi utama GUI voxelizer yang mengintegrasikan pemilihan file OBJ, pengaturan depth, eksekusi voxelisasi asynchronous, preview original/voxel, dan penampilan statistik hasil.

2.10 File main.cpp

main.cpp berfungsi sebagai titik masuk program (entry point) yang menangani parsing argumen command line, validasi input awal, pengumpulan kandidat file OBJ, dan pemanggilan GUI voxelizer. File ini memastikan aplikasi hanya dijalankan ketika parameter dan sumber data valid.

Nama Struktur	Deskripsi
CliOptions	Struktur penyimpanan opsi command line: inputPath, depth, dan smoothNormals.
CliOptions::inputPath	Path file OBJ input yang diberikan pengguna dari command line.
CliOptions::depth	Kedalaman octree untuk proses voxelisasi (default 5, minimum dijaga tidak negatif).
CliOptions::smoothNormals	Flag untuk mengaktifkan mode smooth normal saat viewer mulai dijalankan (--smooth).
printUsage(const string& exeName)	Menampilkan panduan penggunaan aplikasi dan daftar opsi command line yang didukung.
parseCli(int argc, char** argv, CliOptions& options)	Mem-parsing argumen command line (--depth, --smooth, dan path OBJ), serta melakukan validasi format argumen.
main(int argc, char** argv)	Alur utama program: parsing CLI, menampilkan usage jika input tidak valid, mengumpulkan kandidat OBJ valid, lalu menjalankan runVoxelizerGui(...).

BAB 3

SOURCE CODE PROGRAM

3.1 Repository Program

Berikut adalah pranala ke *repository* program:
https://github.com/Dikanugraha-hub/Tucil2_13524127_13524132

3.2 Struktur *Source Code*

3.2.1 types.h

```
#ifndef TYPES_H
#define TYPES_H

#include <vector>
#include <string>
#include <mutex>

using namespace std;

struct Vertex {
    float x, y, z;
};

struct Face {
    int v1, v2, v3;
};

struct MyBoundingBox {
    Vertex minPoint;
    Vertex maxPoint;
};

struct VoxelizationResult {
    vector<Vertex> originalVertices;
    vector<Face> originalFaces;
    vector<Vertex> voxelVertices;
    vector<Face> voxelFaces;
    MyBoundingBox rootBox;
    string outputPath;
    int totalVoxels = 0;
    double elapsedSeconds = 0.0;
};

struct OctreeStats {
    vector<int> nodesCreatedPerDepth;
    vector<int> nodesPrunedPerDepth;
```

```
    mutex mtx;

    void init(int maxDepth) {
        nodesCreatedPerDepth.assign(maxDepth + 1, 0);
        nodesPrunedPerDepth.assign(maxDepth + 1, 0);
    }
};

extern OctreeStats stats;

struct OctreeNode {
    MyBoundingBox box;
    int depth;
    bool isLeaf;
    bool isOccupied;
    OctreeNode* children[8];

    OctreeNode(MyBoundingBox b, int d)
        : box(b), depth(d), isLeaf(true), isOccupied(false) {
        for (int i = 0; i < 8; i++) children[i] = nullptr;
    }

    ~OctreeNode() {
        for (int i = 0; i < 8; i++) delete children[i];
    }
};

#endif
```

3.2.2 octree.h

```
#ifndef OCTREE_H
#define OCTREE_H

#include "types.h"
#include <atomic>

MyBoundingBox computeBoundingBox(const vector<Vertex>& vertices);

bool triangleBoxIntersect(const Vertex& v0, const Vertex& v1,
const Vertex& v2, const MyBoundingBox& box);

bool meshIntersectsBox(const MyBoundingBox& box, const
vector<Vertex>& vertices, const vector<Face>& faces);

void buildOctree(
    OctreeNode* node,
    const vector<Vertex>& vertices,
    const vector<Face>& faces,
```

```
    int maxDepth,
    const std::atomic_bool* cancelToken = nullptr
);

void extractVoxels(
    OctreeNode* node,
    vector<Vertex>& voxelVertices,
    vector<Face>& voxelFaces,
    int maxDepth,
    const std::atomic_bool* cancelToken = nullptr
);

#endif
```

3.2.3 octree.cpp (Implementasi Pembagian Ruang Menjadi 8 *Child)*

```
MyBoundingBox childBox(const MyBoundingBox& box, int childIndex) {
    Vertex c = {
        (box.minPoint.x + box.maxPoint.x) * 0.5f,
        (box.minPoint.y + box.maxPoint.y) * 0.5f,
        (box.minPoint.z + box.maxPoint.z) * 0.5f
    };

    MyBoundingBox child = box;
    child.minPoint.x = (childIndex & 1) ? c.x : box.minPoint.x;
    child.maxPoint.x = (childIndex & 1) ? box.maxPoint.x : c.x;
    child.minPoint.y = (childIndex & 2) ? c.y : box.minPoint.y;
    child.maxPoint.y = (childIndex & 2) ? box.maxPoint.y : c.y;
    child.minPoint.z = (childIndex & 4) ? c.z : box.minPoint.z;
    child.maxPoint.z = (childIndex & 4) ? box.maxPoint.z : c.z;
    return child;
}
```

3.2.4 octree.cpp (Implementasi Uji Irisan Triangle vs Box)

```
bool triangleBoxIntersect(const Vertex& v0, const Vertex& v1,
const Vertex& v2, const MyBoundingBox& box) {

    Vertex center{
        (box.minPoint.x + box.maxPoint.x) * 0.5f,
        (box.minPoint.y + box.maxPoint.y) * 0.5f,
        (box.minPoint.z + box.maxPoint.z) * 0.5f
    };
    Vertex half{
        (box.maxPoint.x - box.minPoint.x) * 0.5f,
        (box.maxPoint.y - box.minPoint.y) * 0.5f,
        (box.maxPoint.z - box.minPoint.z) * 0.5f
```

```
    };
    if (half.x < 1e-9f || half.y < 1e-9f || half.z < 1e-9f) return
false;

    Vertex tv0 = sub(v0, center);
    Vertex tv1 = sub(v1, center);
    Vertex tv2 = sub(v2, center);

    Vertex e0 = sub(tv1, tv0);
    Vertex e1 = sub(tv2, tv1);
    Vertex e2 = sub(tv0, tv2);

    const array<Vertex, 3> boxAxes = {Vertex{1.0f, 0.0f, 0.0f},
Vertex{0.0f, 1.0f, 0.0f}, Vertex{0.0f, 0.0f, 1.0f}};
    const array<Vertex, 3> edges = {e0, e1, e2};

    for (const auto& e : edges) {
        for (const auto& a : boxAxes) {
            Vertex axis = cross(e, a);
            if (!satAxisOverlap(axis, tv0, tv1, tv2, half)) return
false;
        }
    }

    if (!satAxisOverlap(boxAxes[0], tv0, tv1, tv2, half)) return
false;
    if (!satAxisOverlap(boxAxes[1], tv0, tv1, tv2, half)) return
false;
    if (!satAxisOverlap(boxAxes[2], tv0, tv1, tv2, half)) return
false;

    Vertex normal = cross(e0, e1);
    if (!satAxisOverlap(normal, tv0, tv1, tv2, half)) return
false;

    return true;
}
```

3.2.5 octree.cpp (Implementasi Rekursi Utama Octree dan Pruning)

```
void buildOctreeFiltered(
    OctreeNode* node,
    const vector<TriangleData>& triangles,
    const vector<int>& candidateFaces,
    int maxDepth,
    const std::atomic_bool* cancelToken
) {
    if (!node) return;
    if (cancelToken && cancelToken->load()) {
```

```
        node->isLeaf = true;
        node->isOccupied = false;
        return;
    }

    vector<int> intersectingFaces;
    intersectingFaces.reserve(candidateFaces.size());
    for (int faceIndex : candidateFaces) {
        if (cancelToken && cancelToken->load()) {
            node->isLeaf = true;
            node->isOccupied = false;
            return;
        }
        const TriangleData& tri = triangles[faceIndex];
        if (!boxesOverlap(tri.bounds, node->box)) continue;
        if (triangleBoxIntersect(tri.v0, tri.v1, tri.v2,
node->box)) {
            intersectingFaces.push_back(faceIndex);
        }
    }
    if (intersectingFaces.empty()) {
        node->isLeaf = true;
        node->isOccupied = false;
        return;
    }

    node->isOccupied = true;
    if (node->depth >= maxDepth) {
        node->isLeaf = true;
        return;
    }

    array<MyBoundingBox, 8> childBoxes;
    array<vector<int>, 8> childFaceLists;
    bool anyChildOccupied = false;

    for (int i = 0; i < 8; ++i) {
        childBoxes[i] = childBox(node->box, i);
    }

    for (int faceIndex : intersectingFaces) {
        const MyBoundingBox& triBox = triangles[faceIndex].bounds;
        for (int c = 0; c < 8; ++c) {
            if (boxesOverlap(triBox, childBoxes[c])) {
                childFaceLists[c].push_back(faceIndex);
                anyChildOccupied = true;
            }
        }
    }
```

```
    if (!anyChildOccupied) {
        node->isLeaf = true;
        return;
    }

    node->isLeaf = false;
    for (int c = 0; c < 8; ++c) {
        if (cancelToken && cancelToken->load()) {
            node->isLeaf = true;
            node->isOccupied = false;
            return;
        }
        if (childFaceLists[c].empty()) {
            incrementPruned(node->depth + 1);
            continue;
        }
        node->children[c] = new OctreeNode(childBoxes[c],
node->depth + 1);
        incrementCreated(node->depth + 1);
        buildOctreeFiltered(node->children[c], triangles,
childFaceLists[c], maxDepth, cancelToken);
    }
}
```

3.2.6 octree.cpp (Implementasi Inisialisasi Data Kandidat Sebelum Rekursi)

```
void buildOctree(
    OctreeNode* node,
    const vector<Vertex>& vertices,
    const vector<Face>& faces,
    int maxDepth,
    const std::atomic_bool* cancelToken
) {
    if (!node) return;
    if (cancelToken && cancelToken->load()) return;
    stats.init(maxDepth);
    incrementCreated(node->depth);
    vector<TriangleData> triangles;
    vector<int> candidates;
    triangles.reserve(faces.size());
    candidates.reserve(faces.size());

    for (size_t i = 0; i < faces.size(); ++i) {
        if (cancelToken && cancelToken->load()) return;
        const Face& f = faces[i];
        int i1 = f.v1 - 1;
        int i2 = f.v2 - 1;
        int i3 = f.v3 - 1;
        if (i1 < 0 || i2 < 0 || i3 < 0) continue;
```

```
        if (i1 >= static_cast<int>(vertices.size())
            || i2 >= static_cast<int>(vertices.size())
            || i3 >= static_cast<int>(vertices.size())) continue;
        TriangleData tri{};
        tri.v0 = vertices[i1];
        tri.v1 = vertices[i2];
        tri.v2 = vertices[i3];
        tri.bounds = triangleBounds(tri.v0, tri.v1, tri.v2);
        triangles.push_back(tri);
        candidates.push_back(static_cast<int>(triangles.size()) -
1);
    }

    buildOctreeFiltered(node, triangles, candidates, maxDepth,
cancelToken);
}
```

3.2.7 Octree.cpp (Implementasi Ekstraksi Voxel dari Leaf)

```
void extractVoxels(
    OctreeNode* node,
    vector<Vertex>& voxelVertices,
    vector<Face>& voxelFaces,
    int maxDepth,
    const std::atomic_bool* cancelToken
) {
    if (!node) return;
    if (cancelToken && cancelToken->load()) return;
    if (!node->isOccupied) return;

    if (node->isLeaf || node->depth >= maxDepth) {
        appendCube(node->box, voxelVertices, voxelFaces);
        return;
    }

    for (int i = 0; i < 8; ++i) {
        if (cancelToken && cancelToken->load()) return;
        extractVoxels(node->children[i], voxelVertices,
voxelFaces, maxDepth, cancelToken);
    }
}
```

3.2.8 Voxelizer.h

```
#ifndef VOXELIZER_H
#define VOXELIZER_H

#include "types.h"
```

```
#include <atomic>

struct VoxelizerConfig {
    int maxDepth = 5;
    bool useAsync = true;
    const std::atomic_bool* cancelToken = nullptr;
};

VoxelizationResult voxelizeMesh(
    const vector<Vertex>& vertices,
    const vector<Face>& faces,
    const VoxelizerConfig& config
);

#endif
```

3.2.9 voxelizer.cpp

```
#include "voxelizer.h"
#include "octree.h"
#include <chrono>
#include <future>

VoxelizationResult voxelizeMesh(
    const vector<Vertex>& vertices,
    const vector<Face>& faces,
    const VoxelizerConfig& config
) {
    VoxelizationResult result;
    result.originalVertices = vertices;
    result.originalFaces = faces;
    const std::atomic_bool* cancelToken = config.cancelToken;

    if (vertices.empty() || faces.empty()) {
        return result;
    }

    auto t0 = chrono::high_resolution_clock::now();
    result.rootBox = computeBoundingBox(vertices);
    if (cancelToken && cancelToken->load()) {
        result.cancelled = true;
        return result;
    }

    OctreeNode root(result.rootBox, 0);
    if (config.useAsync) {
        auto worker = async(
            launch::async,
```

```
            [&root, &vertices, &faces, &config]() {
                buildOctree(&root, vertices, faces,
config.maxDepth, config.cancelToken);
            }
        );
        worker.get();
    } else {
        buildOctree(&root, vertices, faces, config.maxDepth,
config.cancelToken);
    }

    if (cancelToken && cancelToken->load()) {
        result.cancelled = true;
        auto t1 = chrono::high_resolution_clock::now();
        result.elapsedSeconds = chrono::duration<double>(t1 -
t0).count();
        return result;
    }

    extractVoxels(&root, result.voxelVertices, result.voxelFaces,
config.maxDepth, config.cancelToken);
    if (cancelToken && cancelToken->load()) {
        result.cancelled = true;
    }

    auto t1 = chrono::high_resolution_clock::now();
    result.elapsedSeconds = chrono::duration<double>(t1 -
t0).count();
    result.totalVoxels = static_cast<int>(result.voxelFaces.size()
/ 12);
    return result;
}
```

3.2.10 main.cpp

```
#include "obj_io.h"
#include "viewer.h"
#include <iostream>
#include <string>

namespace {

struct CliOptions {
    string inputPath;
    int depth = 5;
    bool smoothNormals = false;
};

void printUsage(const string& exeName) {
```

```
    cout << "Usage: " << exeName << " [input.obj] [options]\n";
    cout << "Options:\n";
    cout << "  --depth <n>     Kedalaman octree (default: 5)\n";
    cout << "  --smooth        Start viewer dengan smooth
normal\n";
}

bool parseCli(int argc, char** argv, CliOptions& options) {
    for (int i = 1; i < argc; ++i) {
        string arg = argv[i];
        if (arg == "--depth") {
            if (i + 1 >= argc) return false;
            options.depth = stoi(argv[++i]);
            if (options.depth < 0) options.depth = 0;
        } else if (arg == "--smooth") {
            options.smoothNormals = true;
        } else if (!arg.empty() && arg[0] != '-') {
            options.inputPath = arg;
        } else {
            return false;
        }
    }
    return true;
}

}

int main(int argc, char** argv) {
    CliOptions options;
    if (!parseCli(argc, argv, options)) {
        printUsage(argv[0]);
        return 1;
    }

    auto candidates = collectObjCandidates(options.inputPath);
    if (candidates.empty()) {
        cout << "Tidak ditemukan file OBJ yang valid.\n";
        return 1;
    }

    runVoxelizerGui(candidates, options.depth, options.inputPath,
options.smoothNormals);

    return 0;
}
```

BAB 4

ANALISIS ALGORITMA

4.1 Analisis Kompleksitas Waktu

Kompleksitas waktu dari algoritma *voxelization* berbasis *Octree* ini sangat bergantung pada dua variabel utama, yaitu jumlah bidang poligon atau segitiga pada model 3D masukan (direpresentasikan sebagai *N*) dan batas tingkat kedalaman maksimum pohon (direpresentasikan sebagai *D*). Pada setiap simpul di suatu tingkat kedalaman *d*, algoritma harus mengevaluasi interseksi antara *bounding box* simpul tersebut dengan daftar poligon yang diwariskan dari simpul induknya menggunakan *Separating Axis Theorem* (SAT).

Dalam skenario terburuk, yaitu ketika struktur model 3D sangat kompleks dan menyebar sehingga memenuhi seluruh ruang kotak pembatas utama tanpa ada ruang kosong, setiap poligon akan berpotongan dengan setiap sub-ruang. Jika kondisi ini terjadi, maka pada kedalaman ke-d, algoritma akan melakukan evaluasi sebanyak 8^d simpul, dan di setiap simpulnya memeriksa *N* buah poligon. Total operasi secara keseluruhan akan membentuk deret geometri $\sum_{d=0}^{D} N \cdot 8^d$. Oleh karena itu, kompleksitas waktu asimtotik batas atas (upper bound) untuk kasus terburuk ini adalah $O(N \cdot 8^D)$.

Namun, berkat penerapan paradigma *Divide and Conquer*, kompleksitas waktu aktual pada implementasi program ini jauh lebih efisien dari batas terburuknya. Hal ini dicapai melalui mekanisme pemangkasan cabang (*pruning*). Ketika sebuah ruang kubus dievaluasi dan divalidasi tidak memiliki interseksi sama sekali dengan geometri objek (ruang kosong), algoritma akan langsung menghentikan penelusuran rekursif pada cabang tersebut. Pemangkasan ini secara drastis mengurangi ukuran *N* yang harus diperiksa pada tingkat kedalaman selanjutnya, sehingga waktu komputasi difokuskan secara eksklusif hanya pada area permukaan objek yang relevan.

4.2 Analisis Kompleksitas Ruang

Salah satu keunggulan dari paradigma *Divide and Conquer* adalah kemudahan dan kecocokannya untuk dieksekusi secara paralel pada sistem komputasi modern *multi-core*. Pada program ini, optimalisasi konkurensi diimplementasikan secara eksplisit menggunakan mekanisme pemrosesan asinkron (*std::async*) dari pustaka standar C++. Pendekatan ini memungkinkan pelepasan beban komputasi utama, yaitu pembangunan struktur pohon rekursif dan pengujian interseksi poligon yang intensif, ke dalam *worker thread* yang terpisah dari *thread* antarmuka utama. Pemisahan *thread* ini memastikan program dapat terus berjalan secara responsif tanpa mengalami *freezing*, sekaligus mengizinkan interupsi yang aman melalui pemantauan *cancel token* secara *real-time*.

Secara teoritis, karena setiap ruang kubus anak dari sebuah *octant* menempati kuadran spasial yang mutlak terisolasi satu sama lain, evaluasi untuk masing-masing cabang dapat dilakukan sepenuhnya secara independen tanpa menimbulkan masalah bentrokan data. Untuk menangani pencatatan metrik program secara terpusat dari proses yang berjalan paralel, program ini menerapkan sinkronisasi kunci menggunakan *mutex* (*std::mutex*) pada saat memperbarui statistik global, seperti penambahan variabel jumlah simpul yang diciptakan maupun dipangkas. Meskipun penggunaan *mutex* memberikan sedikit *overhead* sinkronisasi, implementasi konkurensi secara keseluruhan memberikan lonjakan percepatan eksekusi waktu nyata yang sangat signifikan jika dibandingkan dengan pengeksekusian sekuensial tradisional.

4.3 Pengaruh Tingkat Kedalaman (*Depth*) terhadap Kinerja dan Resolusi

Kedalaman octree (D) mengontrol resolusi voxel secara langsung. Jika panjang sisi bounding box model adalah (L), maka panjang sisi voxel pada kedalaman (D) kira-kira ($L/2^D$). Artinya, semakin besar depth ukuran voxel semakin kecil dan detail bentuk object semakin baik. Di sisi lain, jumlah kandidat sel ruang bertambah eksponensial, sehingga waktu komputasi dan penggunaan memori ikut meningkat signifikan.

Berdasarkan pengujian, peningkatan depth cenderung memberikan trade-off: Resolusi visual meningkat, tetapi waktu proses, jumlah node terbentuk dan ukuran file output juga meningkat. Pada depth rendah, model hasil voxelisasi lebih kasar tetapi proses cepat, pada depth tinggi, detail lebih halus namun komputasi lebih berat.

Tabel 1. Percobaan menggunakan objek cow

Depth	**Waktu (s)**	**Total Voxel**	**Node Terbentuk**	**Node Pruned**	**Ukuran OBJ Output**
1	0.019	8	8	0	2.72 KB
2	0.025	48	58	14	18.3 KB
3	0.065	236	306	150	97.9 KB
4	0.146	952	1434	910	412 KB
5	0.693	3984	6702	3258	1.8 MB
6	2.285	16420	> 6702	>3258	7.69 MB

4.4 Analisis Kinerja Konkurensi (*Concurrency*)

Implementasi konkurensi pada sistem ini berfokus pada pemisahan komputasi berat dari thread antarmuka. Pada modul GUI, pemuatan preview model original dan proses voxelisasi dijalankan secara asynchronous (std::async), sehingga interaksi pengguna (navigasi kamera, perubahan mode tampilan, input tombol) tetap berjalan selama komputasi berlangsung. Selain itu, sistem menyediakan mekanisme pembatalan berbasis std::atomic_bool (cancelToken) yang diperiksa berkala selama pembentukan octree dan ekstraksi voxel.

Meskipun demikian, konkurensi pada tahap ini belum ditujukan untuk percepatan paralel internal algoritma. Pada alur GUI, proses voxelisasi dipanggil dengan useAsync = false di konfigurasi voxelizer, sehingga pekerjaan inti tetap dieksekusi dalam satu worker background. Artinya, manfaat utama yang diperoleh adalah responsivitas aplikasi (non-blocking UI), bukan speedup komputasi multi-core yang signifikan.

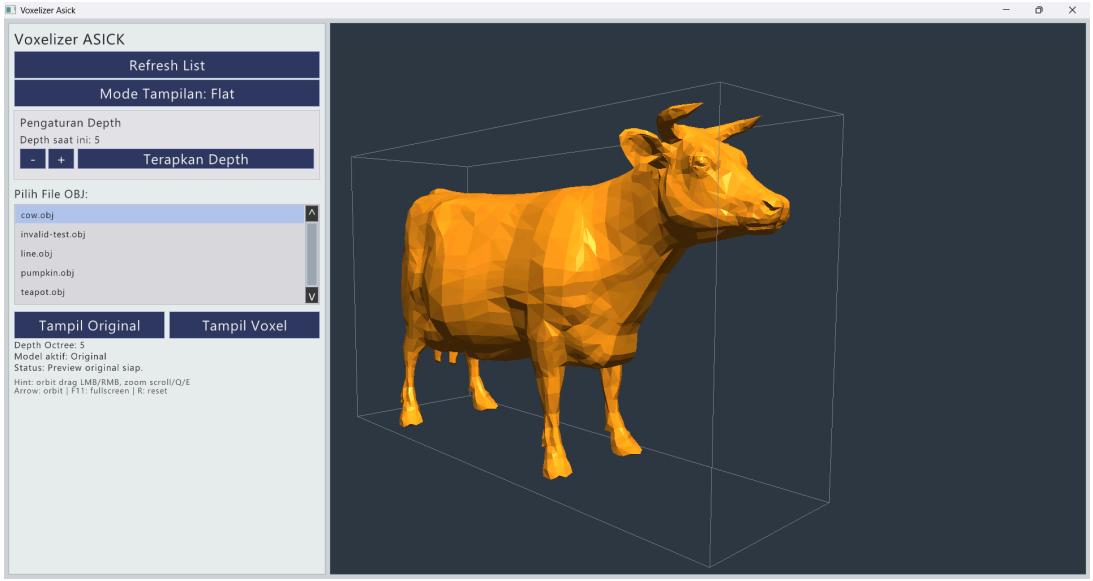
Dari sisi keamanan data, pencatatan statistik octree (nodesCreatedPerDepth dan nodesPrunedPerDepth) dilindungi mutex saat di-update, sehingga mengurangi risiko race condition ketika akses dilakukan dari konteks konkurensi. Secara keseluruhan, desain ini memberikan trade-off yang baik antara kesederhanaan implementasi dan pengalaman pengguna, dengan ruang pengembangan lanjutan berupa paralelisasi subdivision node per-child pada depth tertentu agar throughput komputasi meningkat.

BAB 5

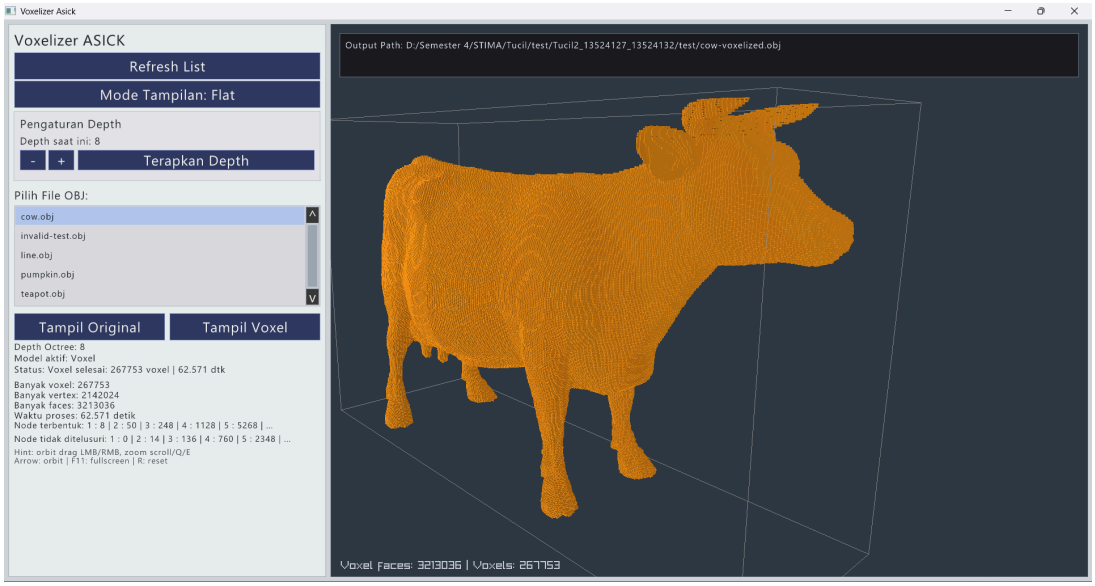
MASUKAN DAN LUARAN PROGRAM

5.1 Test Case 1 (cow.obj)

Data Input (Tampilan Original)

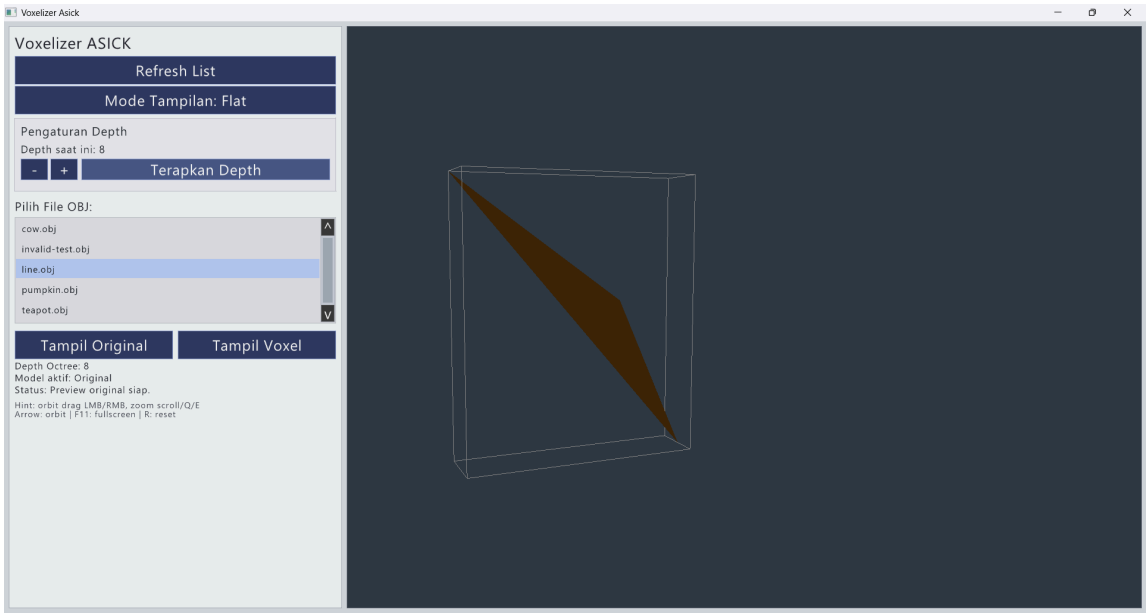


Data Output (Tampilan Voxel)

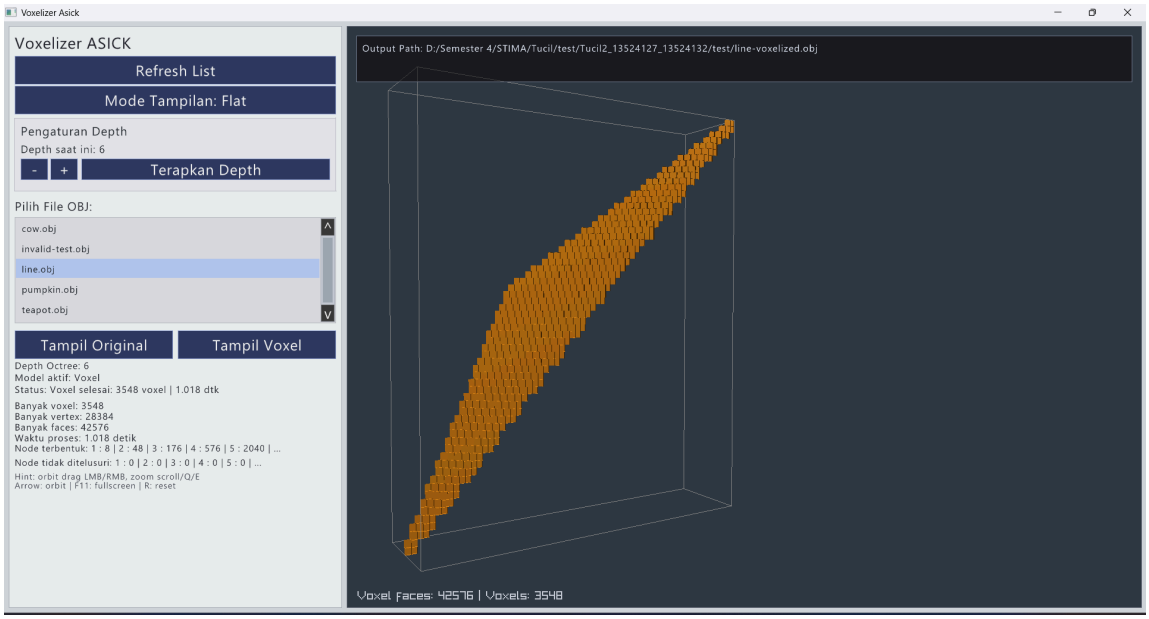


5.2 Test Case 2 (line.obj)

Data Input (Tampilan Original)

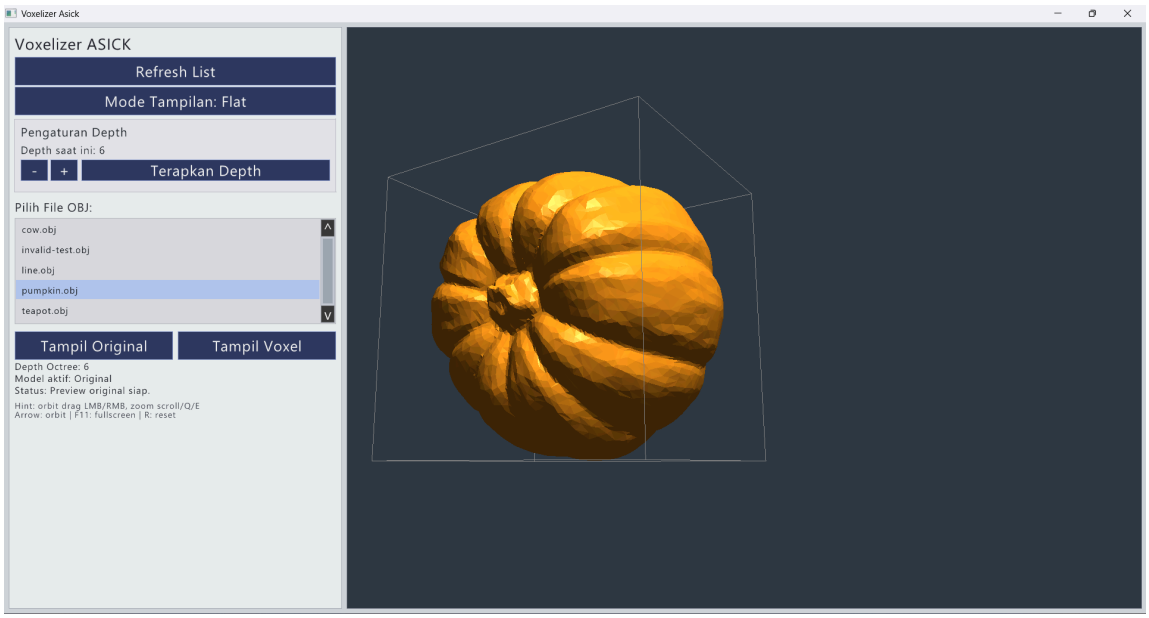


Data Output (Tampilan Voxel)

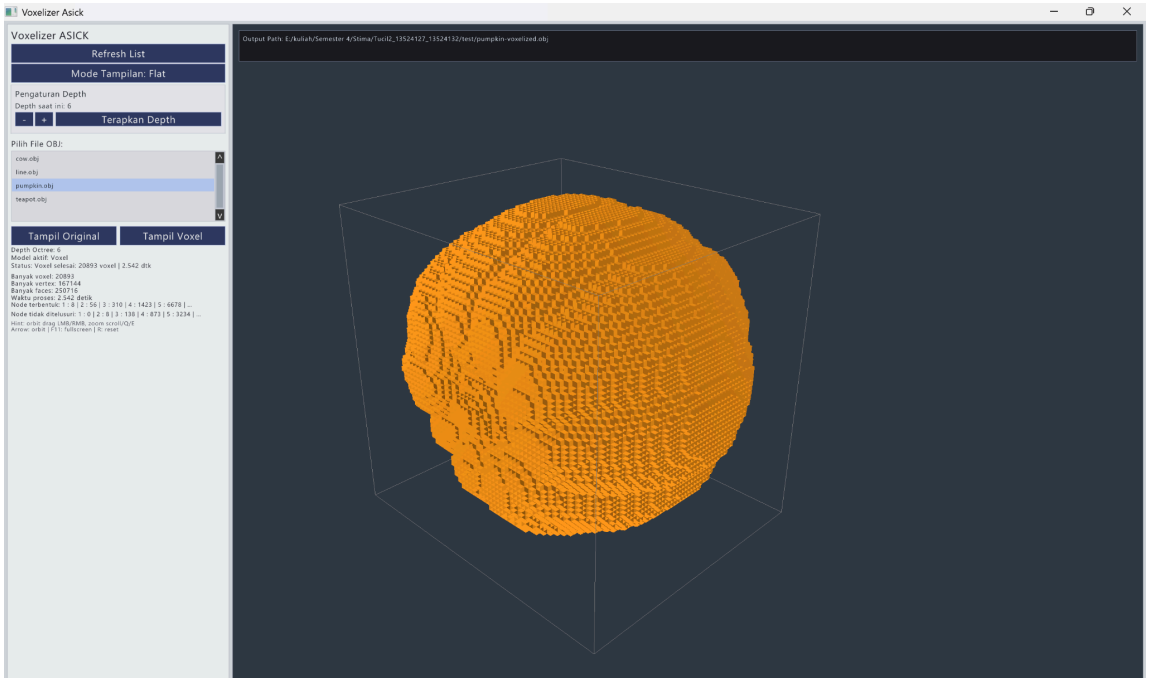


5.3 Test Case 3 (pumpkin.obj)

Data Input (Tampilan Original)

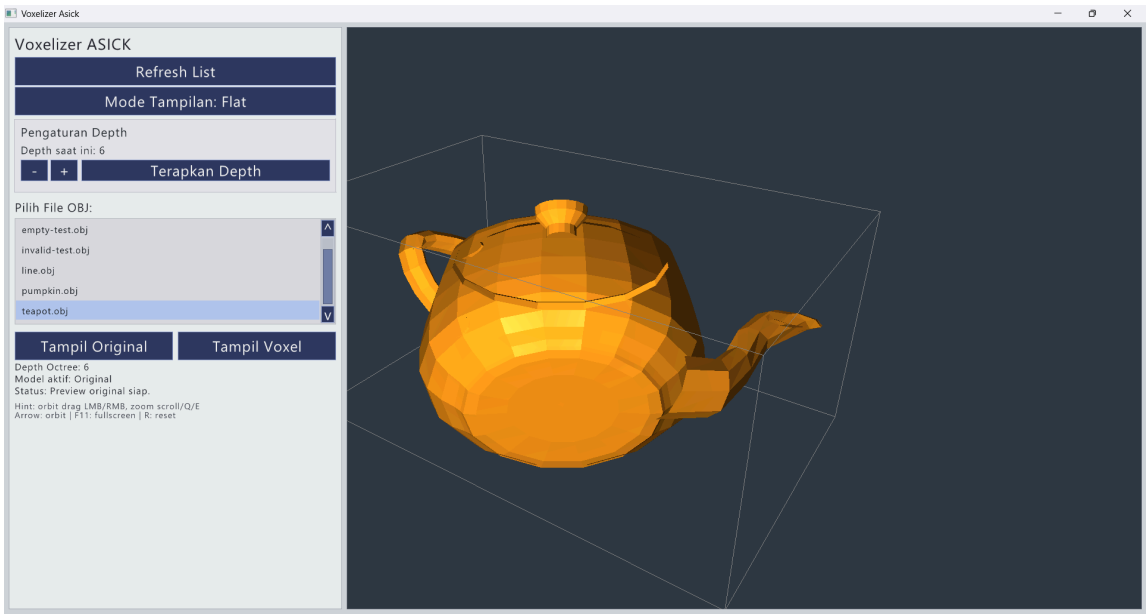


Data Output (Tampilan Voxel)

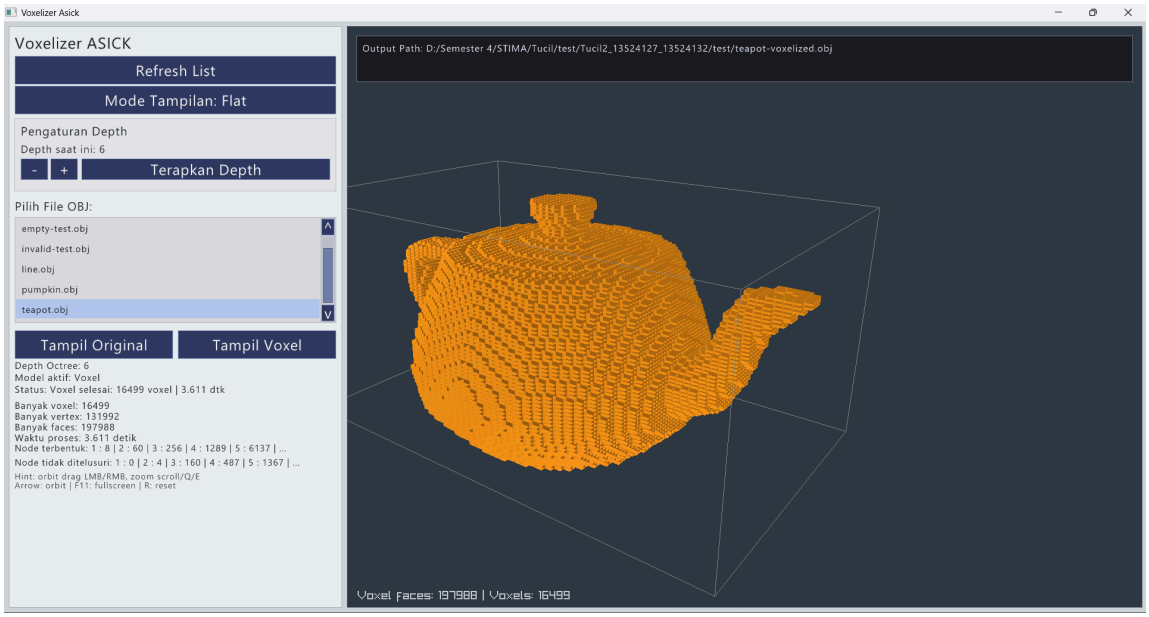


5.4 Test Case 4 (teapot.obj)

Data Input (Tampilan Original)

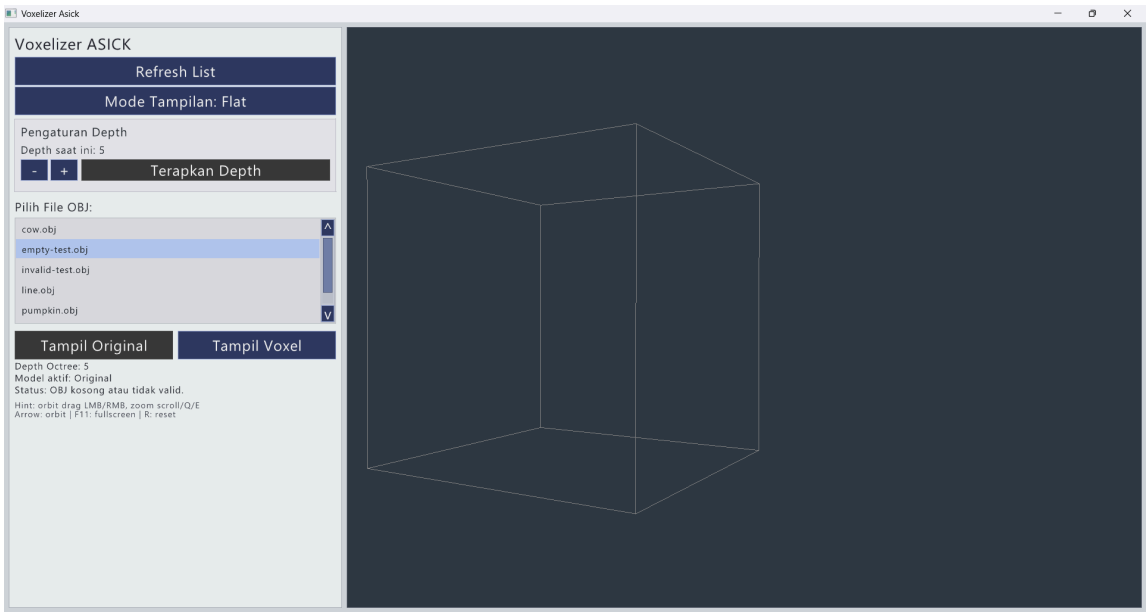


Data Output (Tampilan Voxel)

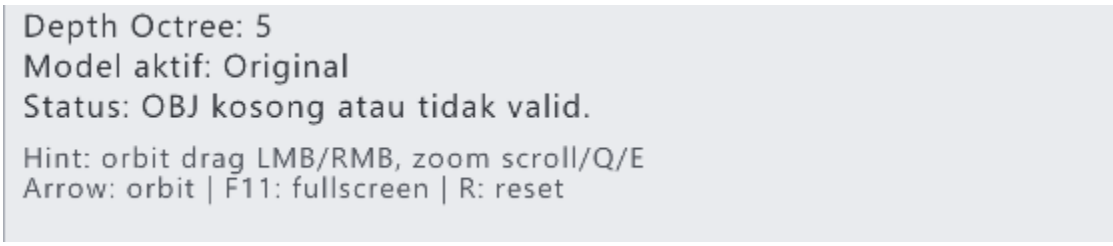


5.5 Test Case 5 (empty-test.obj)

Data Input (Tampilan Original)



Data Output (Tampilan Voxel)



LAMPIRAN

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan *concurrency*	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj	✓	

PERNYATAAN TIDAK MELAKUKAN KECURANGAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Fazri Arrashyi Putra

Dika Pramudya Nugraha

DAFTAR PUSTAKA

[1] https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2023-2024/Algoritma-Divide-and Conquer-(2024)-Bagian1.pdf
[2] GfG. (2024, February 22). Divide and conquer. GeeksforGeeks. https://www.geeksforgeeks.org/divide-and-conquer/#what-is-divide-and-conquer
[3] *Divide and conquer algorithm*. (n.d.). https://www.programiz.com/dsa/divide-and-conquer
[4] Samet, H. (1990). *The Design and Analysis of Spatial Data Structures*. Addison-Wesley.